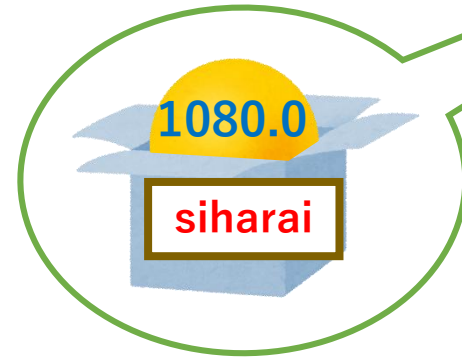
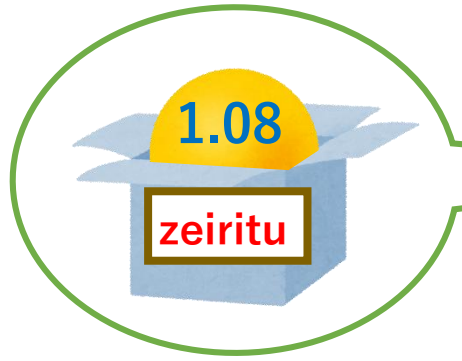


変数の扱い,定数,型



```
class Sample1_04_1 {  
    ^ public static void main (String[] args) {  
        ^  
        ^ int    teika    = 1000 ;           //定価の金額  
        ^  
        ^ double zeiritu = 1.08 ;          //消費税  
        ^  
        ^ double siharai = teika * zeiritu ; //支払い額を計算  
        ^  
        ^ System.out.println(siharai) ;    //支払い額を画面に表示  
        ^  
        ^ }  
    }  
}
```

```
C:\¥Workspace>java Sample1_04_1  
1080.0
```

int teika = 1000 ;

int型の変数である「teika」に1000を入れる

int

整数専用の
変数です！

代入

1000



《変数①》

□変数はデータを1つだけ入れておくことのできる**名前付きの箱**のようなものです。プログラムにおいてデータを扱う際は必ずこの変数が用いられます。
また、変数で扱われるデータのことを**値**と言います。

□変数で扱うデータの種類のことを**型**と言います。
例えば整数を表すint型、文字を表すchar型、小数点数を表すdouble型など、用途にあわせて様々な型が用意されています。

□変数を新しく作ることを**変数宣言**と言い、以下のように行います。

型名 変数名 ;

変数宣言を行うと定義された型・名前の変数が**メモリ上に生成**されます。

□変数に値を入れることを**代入**と言い、以下のように『=』を使用します。イメージしにくいかもしれませんがプログラミングにおいて『=』は「左辺と右辺が等しい」という意味ではなく「右辺の値を左辺の変数に入れる」という意味になります。

変数名 = データ ;

□変数宣言と同時に値を代入することも可能であり、これを**初期化**と言います。

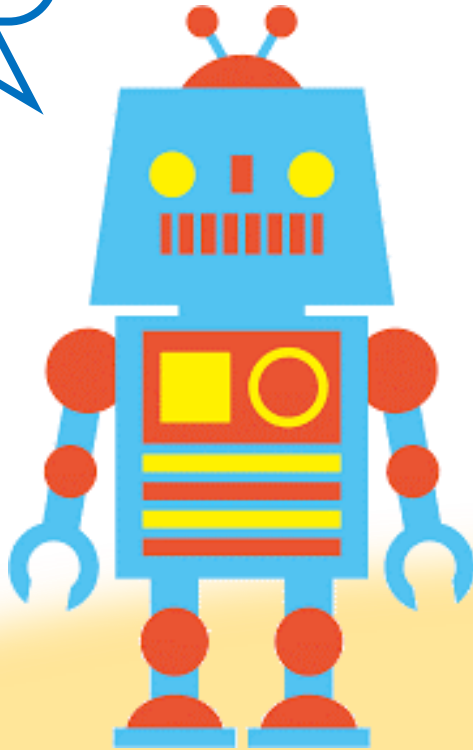
型名 変数名 = 初期データ ;

～メモリとは？～

CPU

(作業者)

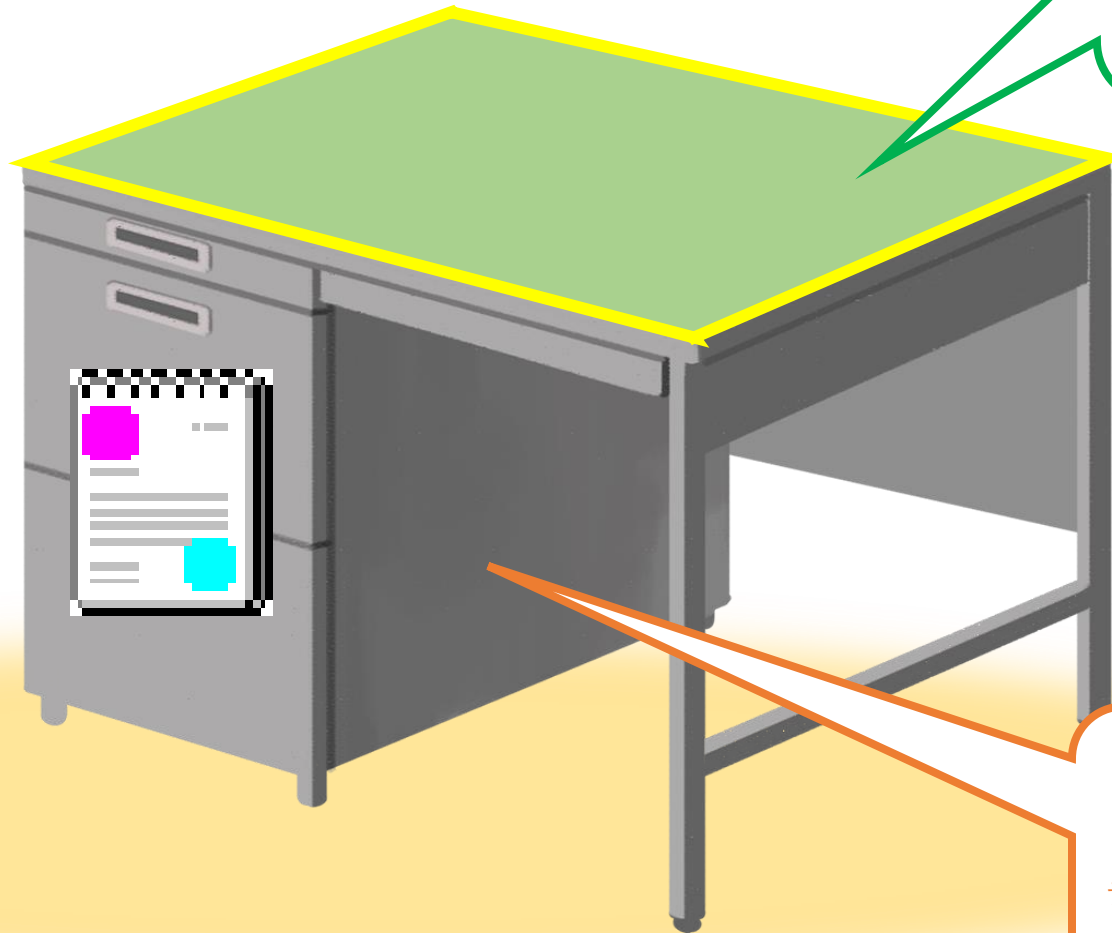
性能が良いほど
処理効率が向上



メモリ

(作業スペース)

処理で扱うデータを
一時的に保存する領域。
大きいほど一度に多くのデータ
を処理することが可能。



HDD

(データ保管)

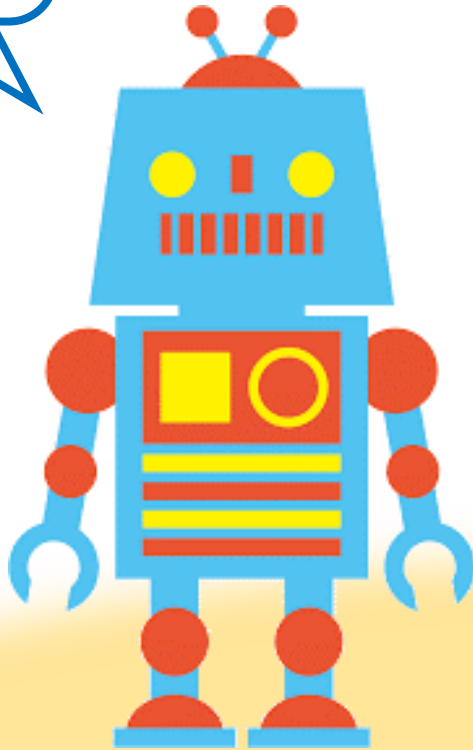
データをずっと保存する領域。
大きいほど多くのデータを
保存することが可能。

～メモリとは？～

CPU

(作業者)

性能が良いほど
処理効率が向上



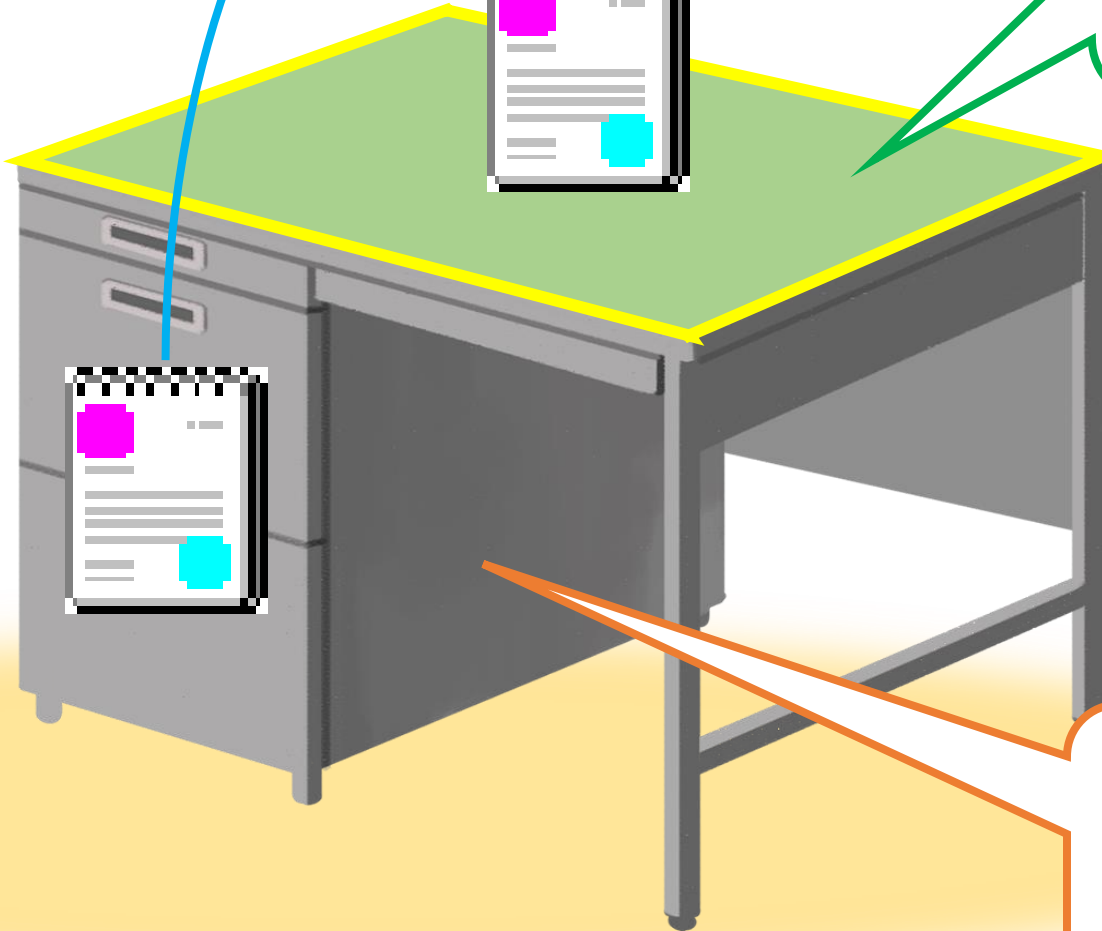
コピー



メモリ

(作業スペース)

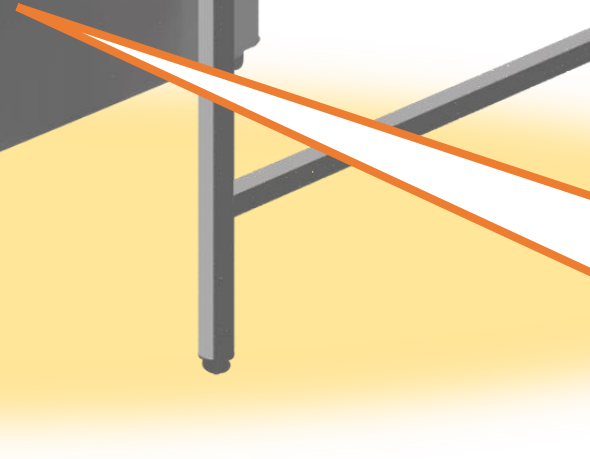
処理で扱うデータを
一時的に保存する領域。
大きいほど一度に多くのデータ
を処理することが可能。



HDD

(データ保管)

データをずっと保存する領域。
大きいほど多くのデータを
保存することが可能。

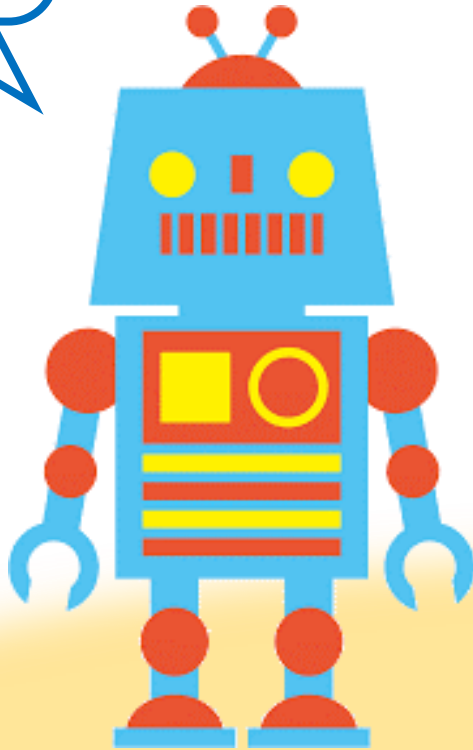


～メモリとは？～

CPU

(作業者)

性能が良いほど
処理効率が向上



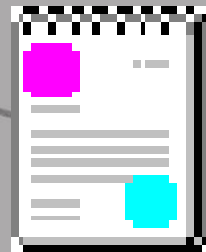
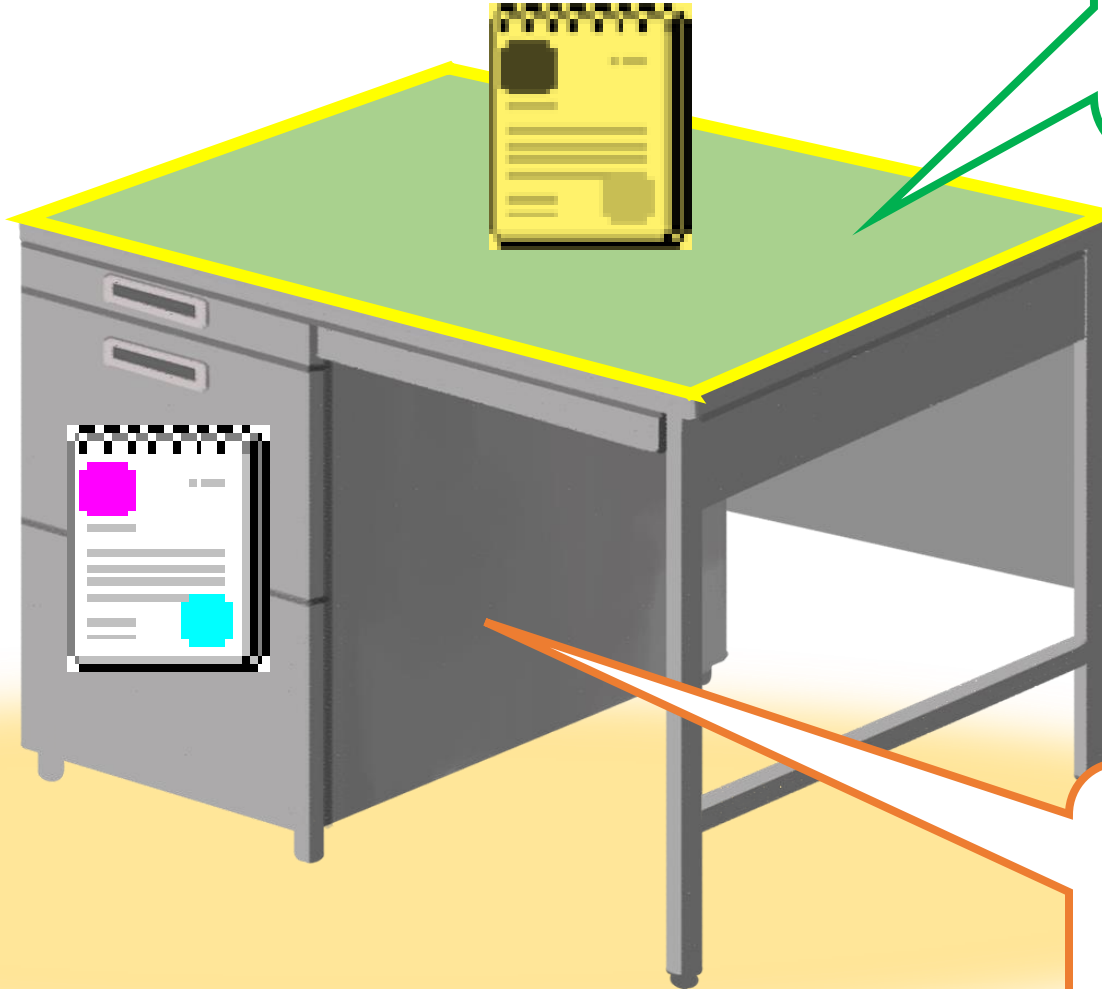
内容変更
(作業)



メモリ

(作業スペース)

処理で扱うデータを
一時的に保存する領域。
大きいほど一度に多くのデータ
を処理することが可能。



HDD

(データ保管)

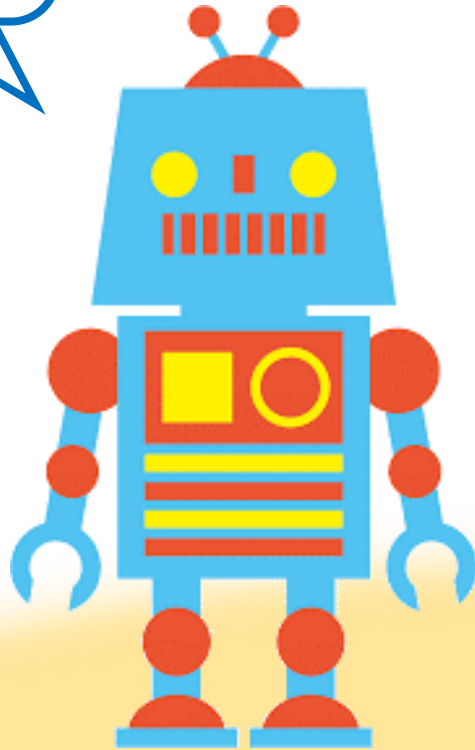
データをずっと保存する領域。
大きいほど多くのデータを
保存することが可能。

～メモリとは？～

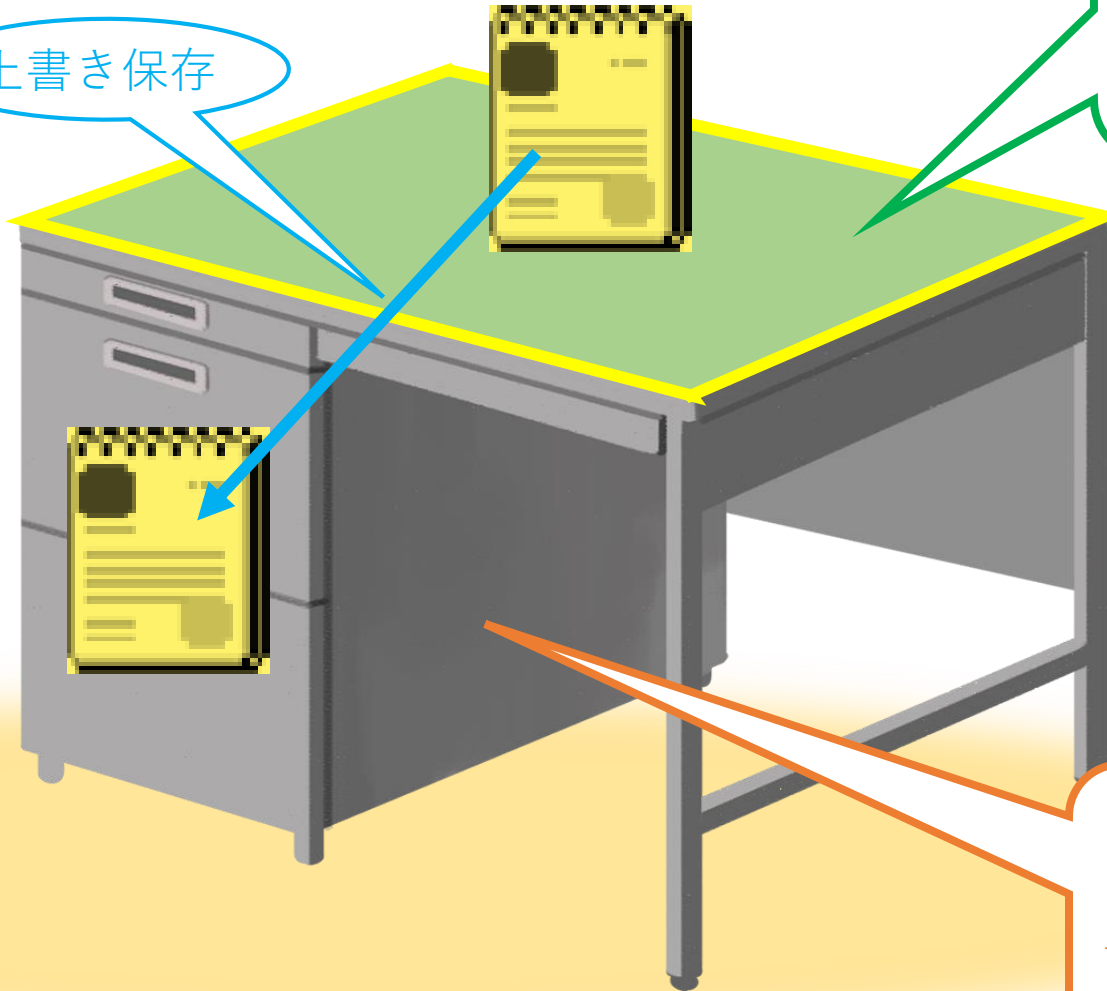
CPU

(作業者)

性能が良いほど
処理効率が向上



上書き保存



メモリ

(作業スペース)

処理で扱うデータを
一時的に保存する領域。
大きいほど一度に多くのデータ
を処理することが可能。

HDD

(データ保管)

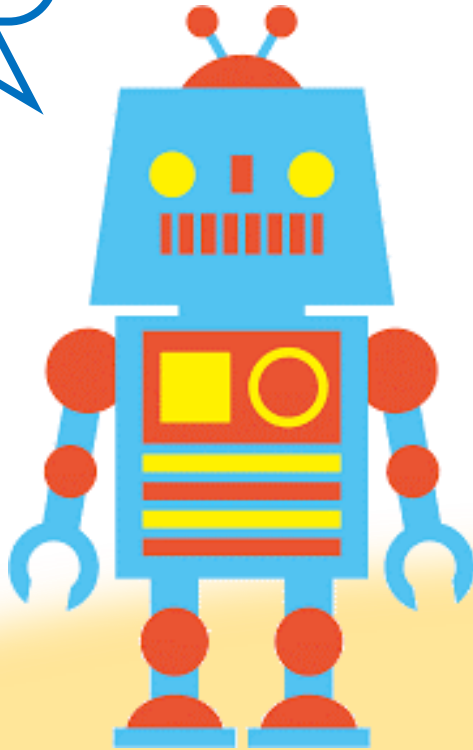
データをずっと保存する領域。
大きいほど多くのデータを
保存することが可能。

～メモリとは？～

CPU

(作業者)

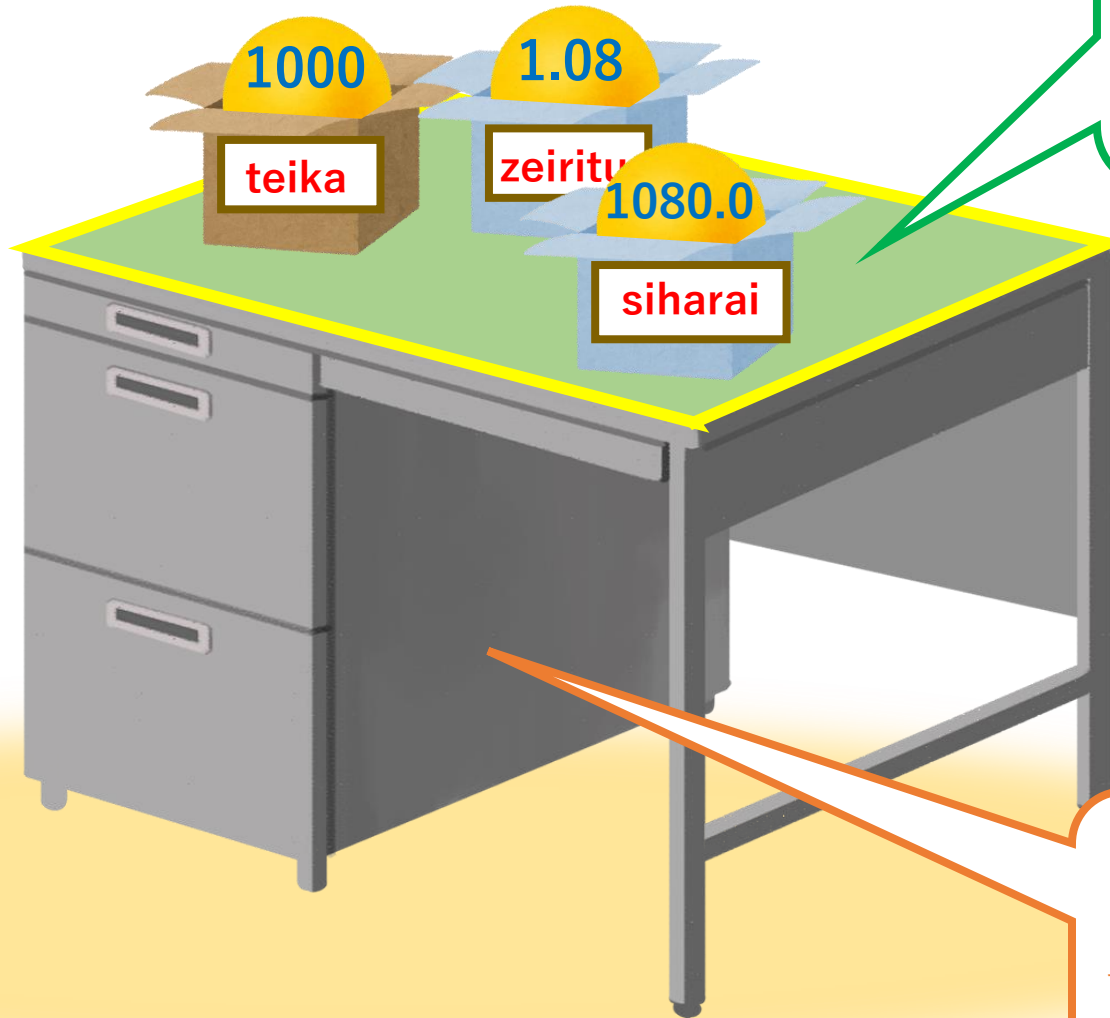
性能が良いほど
処理効率が向上



メモリ

(作業スペース)

処理で扱うデータを
一時的に保存する領域。
大きいほど一度に多くのデータ
を処理することが可能。



HDD

(データ保管)

データをずっと保存する領域。
大きいほど多くのデータを
保存することが可能。

int teika = 1000 ;

int型の変数である「teika」に1000を入れる

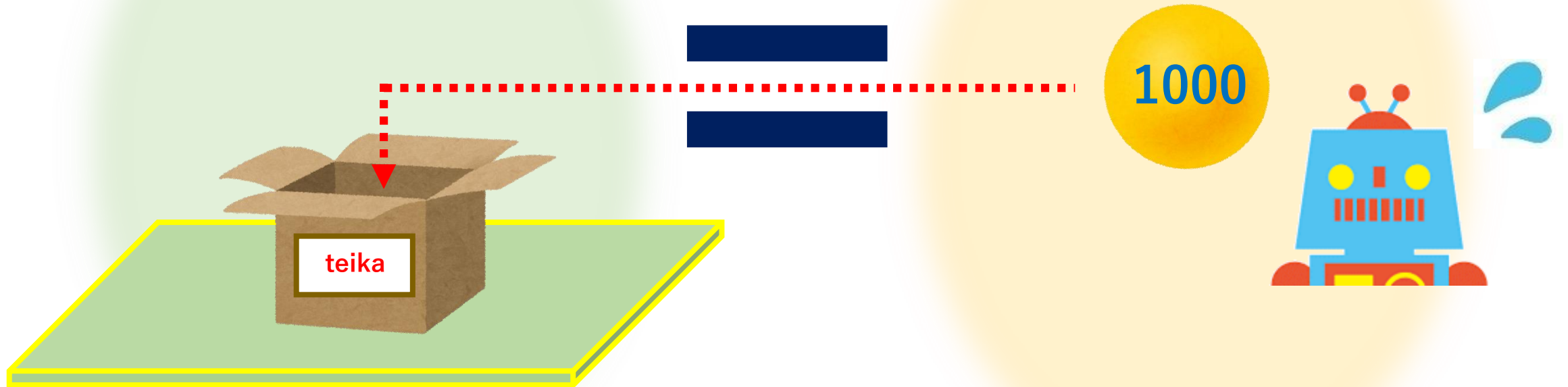
《左辺》

後の処理で使用する値
を一時的に保存しておくために
メモリ上の一部領域（＝変数）を確保

代入

《右辺》

変数に格納するための
値をJVMが演算して準備



STEP 1

入れ物の準備

```
int teika = 1000 ;
```

int型の変数である「teika」に1000を入れる

《左辺》

後の処理で使用する値
を一時的に保存しておくために
メモリ上の一部領域（＝変数）を確保

Javaが確保している
メモリ上の領域
(JVMの作業スペース)

《右辺》

変数に格納するための
値をJVMが演算して準備

STEP 1

入れ物の準備

```
int teika = 1000 ;
```

int型の変数である「teika」に1000を入れる

《左辺》

後の処理で使用する値
を一時的に保存しておくために
メモリ上の一部領域（＝変数）を確保

Int型のデータの
保存に必要な分だけの
メモリ領域を確保



《右辺》

変数に格納するための
値をJVMが演算して準備

STEP 1 `int teika` = 1000 ;

入れ物の準備

int型の変数である「teika」に1000を入れる

《左辺》

後の処理で使用する値
を一時的に保存しておくために
メモリ上の一部領域（＝変数）を確保

確保したメモリ領域に
名前をつける
（変数名）



《右辺》

変数に格納するための
値をJVMが演算して準備

`int teika = 1000;` STEP 2
入れる値の準備

int型の変数である「teika」に1000を入れる

《左辺》

後の処理で使用する値
を一時的に保存しておくために
メモリ上の一部領域（＝変数）を確保



《右辺》

変数に格納するための
値をJVMが演算して準備



`int teika = 1000 ;`

STEP 3

int型の変数である「teika」に1000を入れる 代入の実行

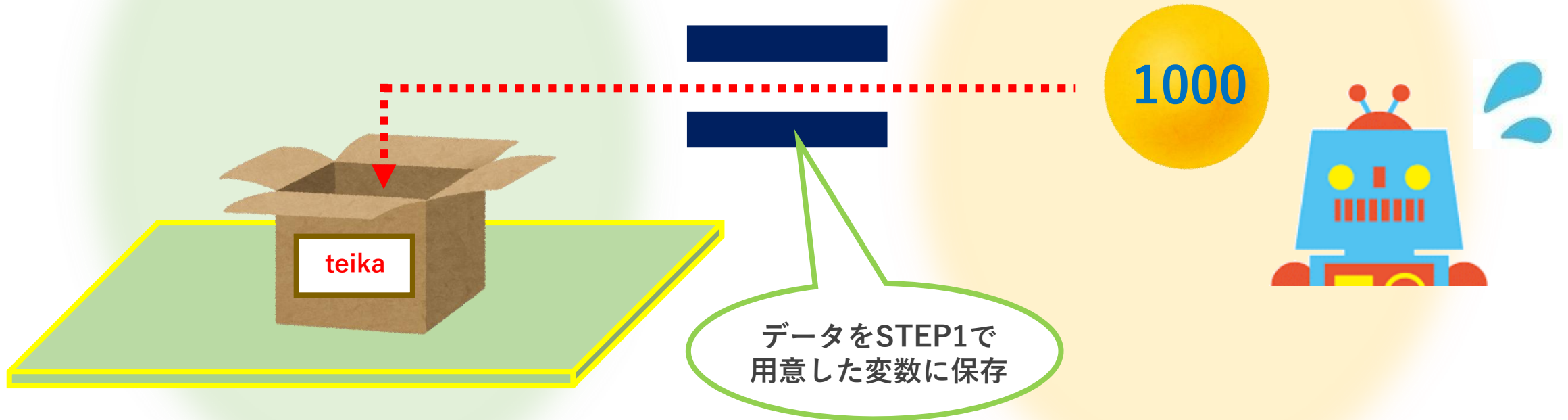
《左辺》

後の処理で使用する値
を一時的に保存しておくために
メモリ上の一部領域（＝変数）を確保

《右辺》

変数に格納するための
値をJVMが演算して準備

代入



int teika = 1000 ;

int型の変数である「teika」に1000を入れる

《左辺》

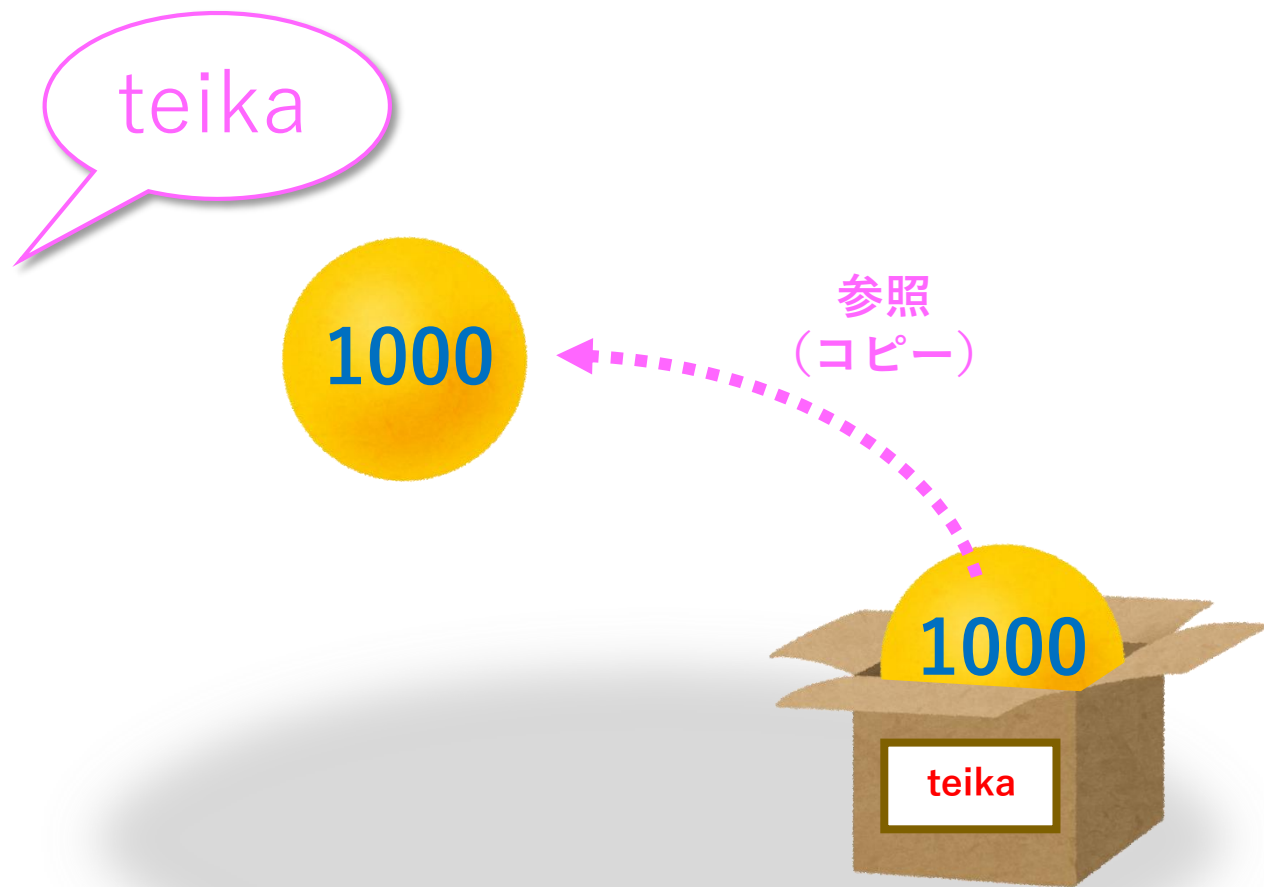
後の処理で使用する値
を一時的に保存しておくために
メモリ上の一部領域（＝変数）を確保

《右辺》

変数に格納するための
値をJVMが演算して準備

右辺のデータが
左辺の変数に
保存された！





《変数②》

- 変数名を指定することで変数内の値を取得することを**参照**と言います。
参照はコピーに近い挙動をするため特に参照先の値に影響は与えません。
- 参照の処理の際、参照先の変数が存在していない場合は当然エラーとなります。
プログラムが上から順番に処理されることを意識し、必ず既に変数宣言されているかを確認しましょう。

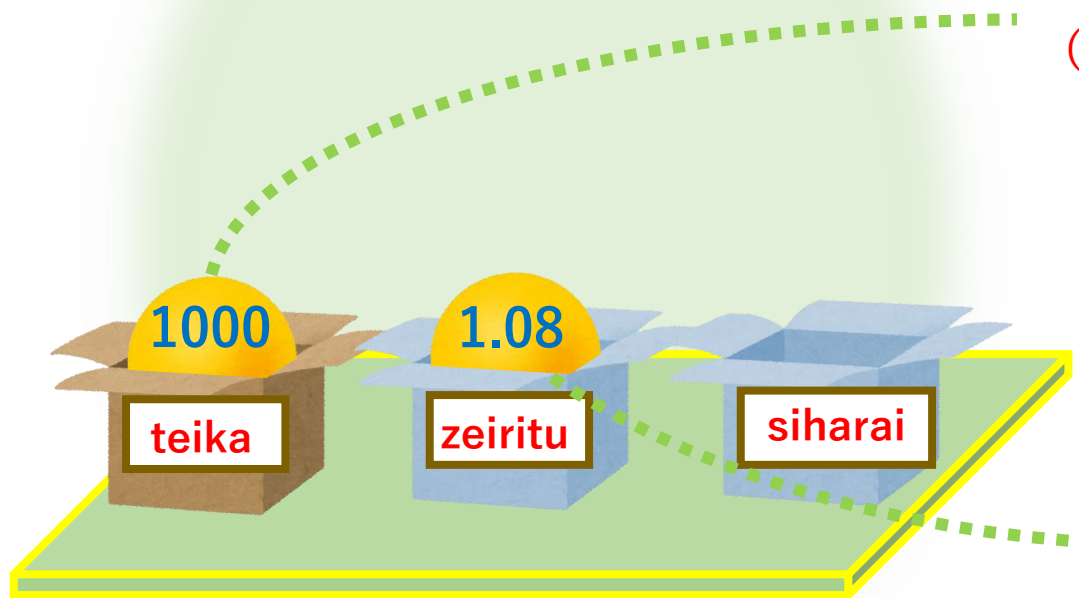
Sample1_04_1.java

```
1  /*-< Sample1_04_1 >-----<
2  定価1000円の商品の税込価格を計算して表示するプログラムです。<
3  消費税は8%で計算しています。<
4  -----*/<
5  class Sample1_04_1 {<
6  ^    public static void main (String[] args) {<
7  ^    ^    <
8  ^    ^    int    teika    = 1000 ;    //定価の金額<
9  ^    ^    <
10 ^    ^    double zeiritu = 1.08 ;    //消費税<
11 ^    ^    <
12 ^    ^    double siharai = teika * zeiritu ;    //支払い額を計算<
13 ^    ^    <
14 ^    ^    System.out.println(siharai) ;    //支払い額を画面に表示<
15 ^    ^    <
16 ^    }<
17 }
```

```
int    teika    = 1000 ;
double zeiritu  = 1.08 ;
double siharai  = teika * zeiritu ;
```

《左辺》

後の処理で使用する値
を一時的に保存しておくために
メモリ上の一部領域（＝変数）を確保

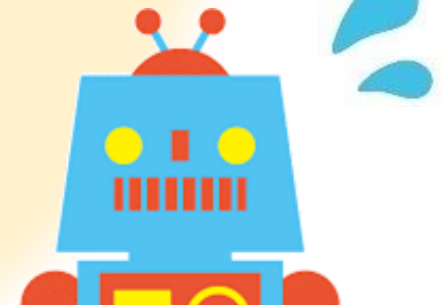


《右辺》

変数に格納するための
値をJVMが演算して準備

参照
(コピー)

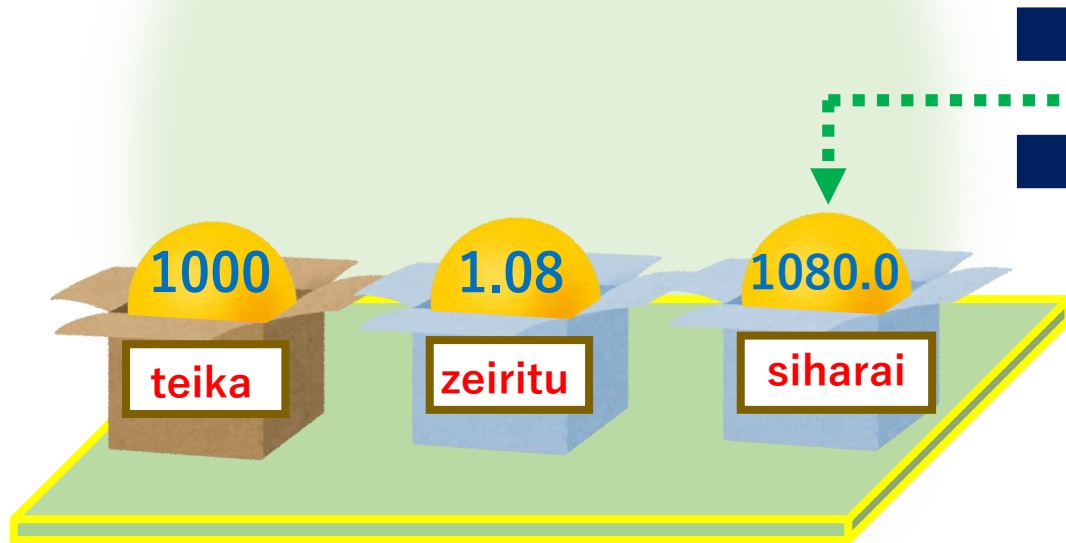
参照
(コピー)



```
int    teika    = 1000 ;  
double zeiritu  = 1.08 ;  
double siharai  = teika * zeiritu ;
```

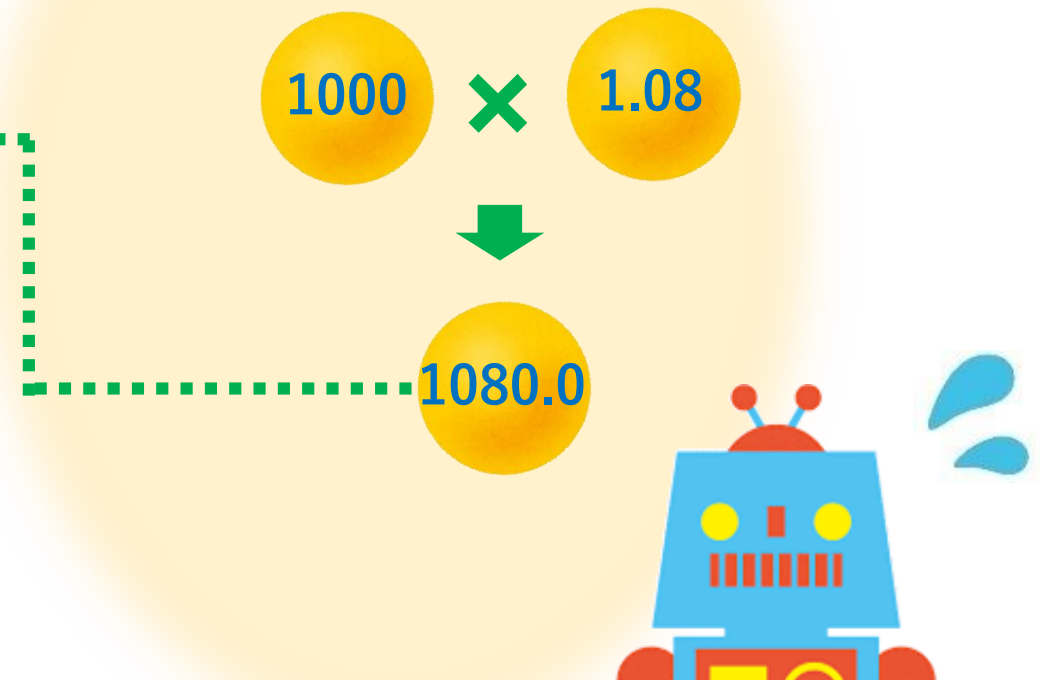
《左辺》

後の処理で使用する値
を一時的に保存しておくために
メモリ上の一部領域（＝変数）を確保



《右辺》

変数に格納するための
値をJVMが演算して準備



```
int teika = 1000 ;  
double zeiritu = 1.08 ;  
double siharai = teika * zeiritu ;
```

イコールより左で指定された変数名は
変数（箱）を表す



《左辺》
箱の世界

イコールより右で指定された変数名は
変数内の値を表す



《右辺》
データの世界

～変数名をつける際の注意～

- ☐ 半角英数と半角アンダーバーのみで記述すること
- ☐ 先頭文字に数字は使用できない
- ☐ 1文字目は必ず小文字
- ☐ 大文字と小文字は区別されることに注意
- ☐ 予約語（Javaの中で特別な意味を持つ単語）は使えない

— 予約語一覧 —

abstract, boolean, break, byte, case, catch, char, class, const, continue,
default, do, double, else, extends, final, finally, float, for, goto, if,
implements, import, instanceof, int, interface, long, native, new, package,
private, protected, public, return, short, static, strictfp, super, switch,
synchronized, this, throw, throws, transient, try, void, volatile, while

～変数名をつける際の心得～

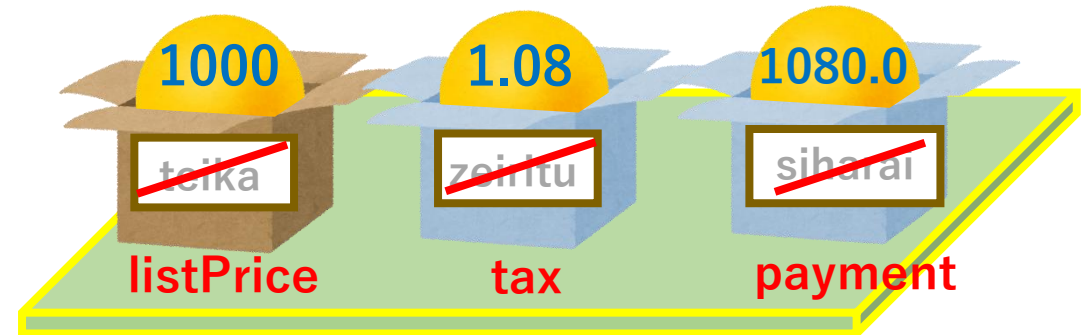
★必ず現場のコーディング規約に従いましょう！（最重要）

★誰が見ても見ただけでどんなデータか
理解できる名前をつけよう！



★できれば英語で！

★単語の区切りは大文字（キャメルケース）で！
（理由があればアンダーバー区切り（スネークケース）でも可）

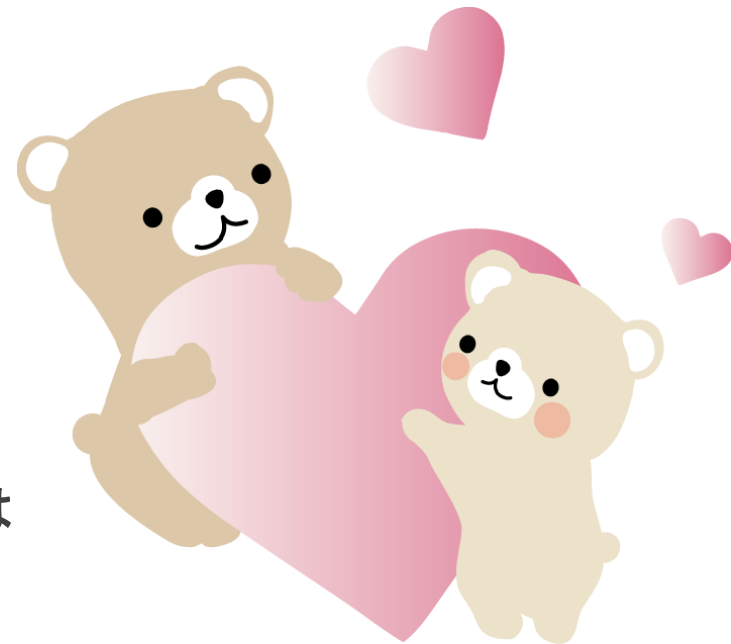


☆以下の自動ネーミングサービスが便利です！（特に英語が苦手な人！）

<codic – デベロッパーのためのネーミング辞書>

<https://codic.jp/>

思いやりあるソースコードを！！



現場で書くソースコードはレビュー・改修・流用などの目的でのちのち他人に触られたり参考にされたりする可能性が非常に高いです。可読性や再利用性の高さ、コメントの親切さ、ソースコードのキレイさはみなさんの他者に対する思いやりの表れです。

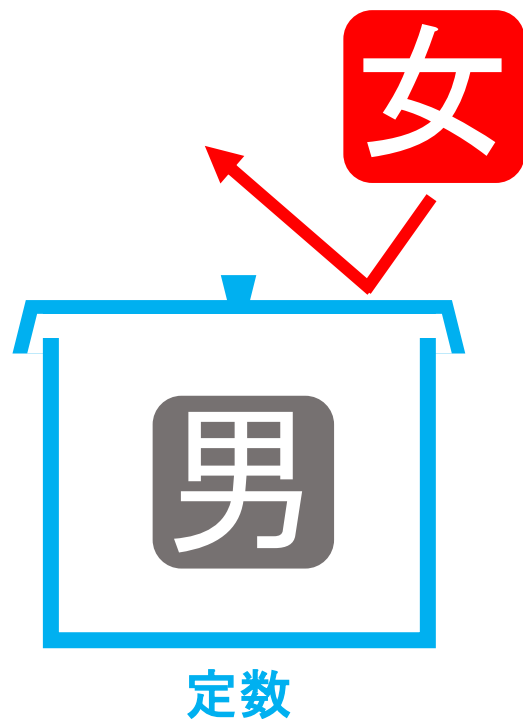
プログラマーの採用シーンではソースコードをチェックされることがよくありますが、自分本位なソースコードを書いていると間違いなく「この人は他者を思いやれない人だな」と判断されるでしょう。

誰かが書いたコードの質は、必ず他の誰かが書くコードの質に影響します。もし私が質の低いコードを書いてしまったとしたらどうでしょうか。その場合は、誰か他の人が非常に質の高いコードを書いていたとしても、私のコードを利用した途端に悪影響を受け、同じレベルにまで質が低下してしまいます。(中略) 大事なものは、コードそのものの質ではなく、他への影響です。他人の存在を意識すれば、他人の書くコードにも当然良い影響を与えることになります。チームの同僚のことを考え、思いやりを持ってコードを書けば、それは同僚たちにとって価値あるコードとなり、いずれ自分にも良い影響となって返ってきます。どんなかたちであれ、そのコードに触れた誰もが、触れる前より良い人間、良いプログラマーになれる、そういうコードを書くようにすべきでしょう。

— Aslam Khan



```
char gender = '男';  
gender = '女';
```



```
final char GENDER = '男';  
GENDER = '女'; ←エラー
```

《変数③（定数）》

- 初期化された状態からデータを変更することのできない変数のことを**定数**と言います。
変数宣言の際に『final』を用いることで定数として扱えるようになります。

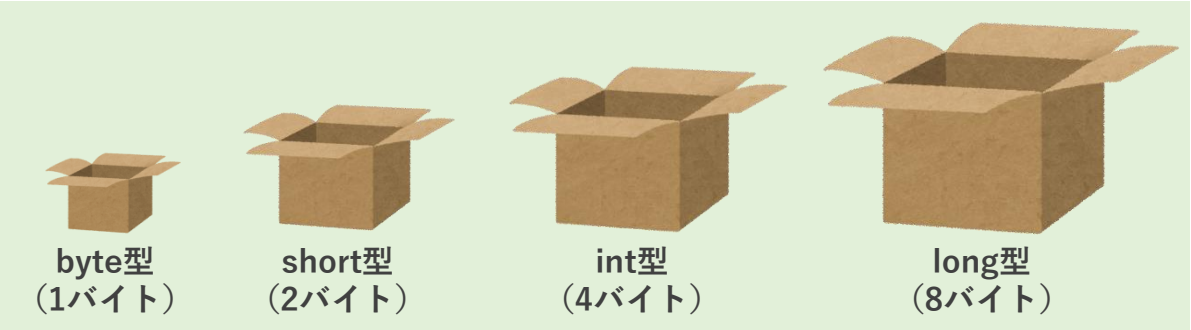
final 型名 定数名 = 初期値（固定値）；

- 代入を可能とする通常の変数と一目見て区別できるよう、**定数名は半角英数の大文字**で記述することが一般的です。
全て大文字で書くため大文字区切り（キャメルケース）は使用できません。
そのため定数名は必ずアンダーバー区切り（スネークケース）で記述します。

▼押さえておくべき型

種類	データ型	扱えるデータ
整数	byte	およそ±128
	short	およそ±3万2千
	int	およそ±21億
	long	およそ±900京
小数点数	float	小数 (doubleより厳密さに欠ける)
	double	小数
文字	char	1文字 (Unicode文字)
	String	文字列
論理値	boolean	trueかfalse

▼違いは箱の「大きさ」 ※小数点数も同じ



≪変数④（型）≫

- 型は「**整数**」「**小数点数**」「**文字**」「**論理値**」の4種類に大別されます。
- 同じ整数を扱う型でも4種類、小数点数を扱う型でも2種類用意されています。
これらの違いはデータ容量（箱の大きさ）にあり、扱う値の大きさに応じて最適な型を選ぶことでメモリを節約することができます。
ただし、現在はメモリに関する技術が飛躍的に向上し、メモリ節約の重要性が下がっているため、扱うデータの大きさに関わらず**整数ならint型、小数点数ならdouble型**を使用するのが一般的になっています。
- 文字を扱う型は2種類あります。**char**は大文字/小文字区別せず1文字のみを扱える型であり、**String**は0文字以上の文字列を扱うことのできる型です。
- boolean**は論理値を扱える型で、格納できるのはtrue(正しい) / false(正しくない) のどちらかのみです。

リテラル
(マジックナンバー)

```
int teika = 1000;
```

整数は暗黙的にint型

▼リテラルと対応する型

データ種類	書式
int	5000000
long	35000000000L ※値の末尾に L を付ける
double	6.05
float	1.35F ※値の末尾に F を付ける
char	'土' ※シングルクォーテーションで囲う
boolean	true あるいは false ※文字列に見えるがダブルコーテーションは使わない
String	"Hello World!!" ※ダブルコーテーションで囲う

≪変数⑤（リテラル）≫

- プログラム中に記述された具体的な値のことを **リテラル** と呼びます。
- 整数リテラルは **暗黙的にint型** として扱われます。
- 小数点数リテラルは **暗黙的にdouble型** として扱われます。
- 文字リテラルは **シングルクォーテーション囲いだとchar型、ダブルコーテーション囲いだとString型** として扱われます。
- 数値リテラルはただそれだけではどんな意味をもった数値か分からずソースコードが難読化してしまうため、極力変数として名前づけして管理することが望ましいです。こうした背景から数値リテラルは「**マジックナンバー**（意味はわからないがコードは動く、魔法の数字）」とも呼ばれ、プログラミングのアンチパターンとされています。

～マジックナンバーを使うのは極力避けましょう！～

数値を直接記述した場合（消費税8%）

```
Double price1 = 98 * 1.08; ← 消費税
Double price2 = 198 * 1.08; ← 消費税
Double price3 = 398 * 1.08; ← 消費税
Double price4 = 999 * 1.08; ← 消費税
Double xxxxxx = 1000 * 1.08; ← 消費税でない
```

数値を直接記述した場合（消費税10%）

```
Double price1 = 98 * 1.10;
Double price2 = 198 * 1.10;
Double price3 = 398 * 1.10;
Double price4 = 999 * 1.10;
Double xxxxxx = 1000 * 1.08;
```

定数を使って記述した場合（消費税8%）

```
final double TAX_RATE = 1.08;
final double XXX_RATE = 1.08;
Double price1 = 98 * TAX_RATE;
Double price2 = 198 * TAX_RATE;
Double price3 = 398 * TAX_RATE;
Double price4 = 999 * TAX_RATE;
Double xxxxxx = 1000 * XXX_RATE;
```

定数を使って記述した場合（消費税10%）

```
final double TAX_RATE = 1.10;
final double XXX_RATE = 1.08;
Double price1 = 98 * TAX_RATE;
Double price2 = 198 * TAX_RATE;
Double price3 = 398 * TAX_RATE;
Double price4 = 999 * TAX_RATE;
Double xxxxxx = 1000 * XXX_RATE;
```

消費税改正

「この数字の意味はわからないが、
とにかくプログラムは正しく動く。まるで魔法の数字だ」

エスケープ文字

エスケープ 文字	意味
¥b	バ ックスペ ース
¥f	改ペ ージ
¥n	改行
¥r	リタ ーン
¥t	タブ

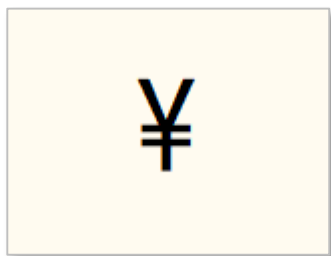
エスケープ 文字	意味
¥'	'
¥"	"
¥¥	¥

文字として表現できない情報は
エスケープシーケンスを用いて表現！

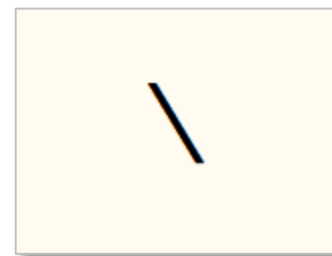
≪変数⑥（エスケープシーケンス）≫

□エスケープシーケンス

タブや改行といった文字として表現できない情報を表現したいときや、Javaの仕様においてあらかじめ**特殊な意味づけのされている記号**（シングルクォーテーションなど）を文字として表現したいときに、**文字の先頭に『¥』（環境によって『\』と表記されることもある）を付けます。**こうした特殊な記述の仕方をエスケープシーケンスと呼び、『¥（または\）』を使用して表現された特殊文字をエスケープ文字と呼びます。

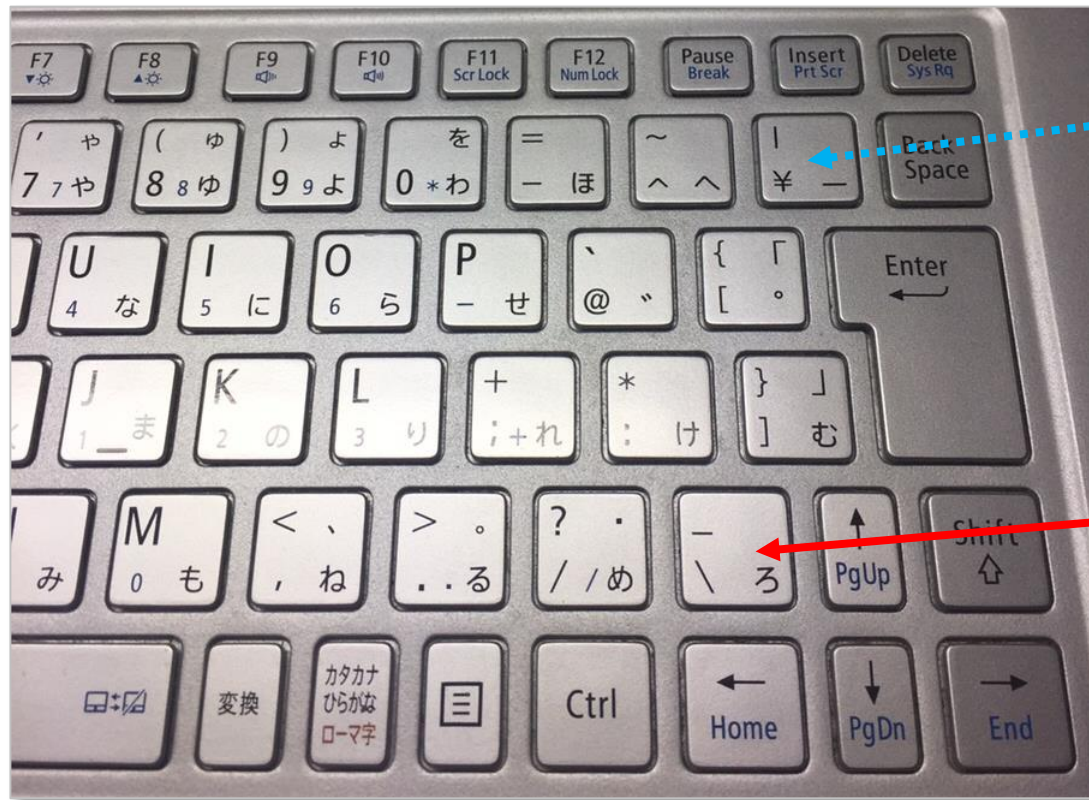


フォント：MSゴシック の場合
(円マーク)



フォント：Osaka の場合
(バックスラッシュ)

< Windowsの場合 >



『¥』の絵柄があるキー
は使用しない

『\』の絵柄があるキー
より入力が可能

< Macの場合 >



『 option 』キーを押しながら
『 ¥ 』の絵柄があるキーを押すことで
入力が可能

<演習：Ex1_05_1>

読む人への
思いやりを持って
書いてね

[illegible]